

Deadlock

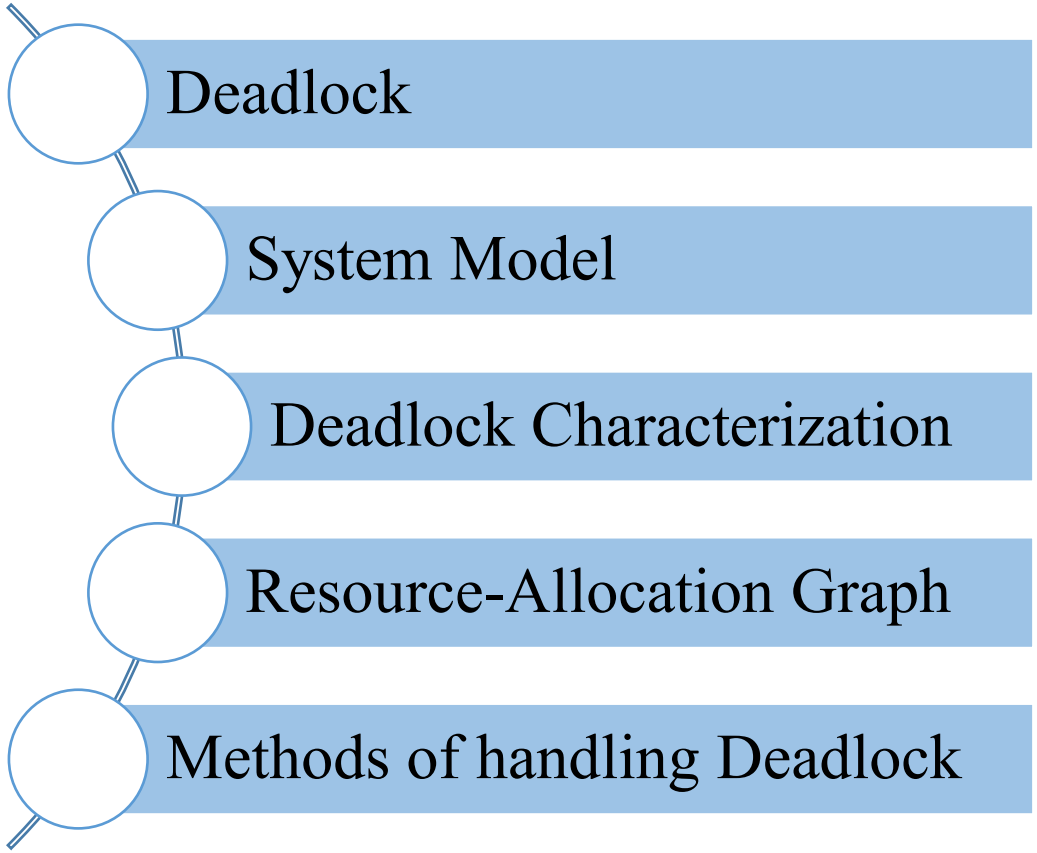
CSE 3331: Operating Systems

Sajid Fardin Dipto
Lecturer
Department of CSE
Northern University Bangladesh

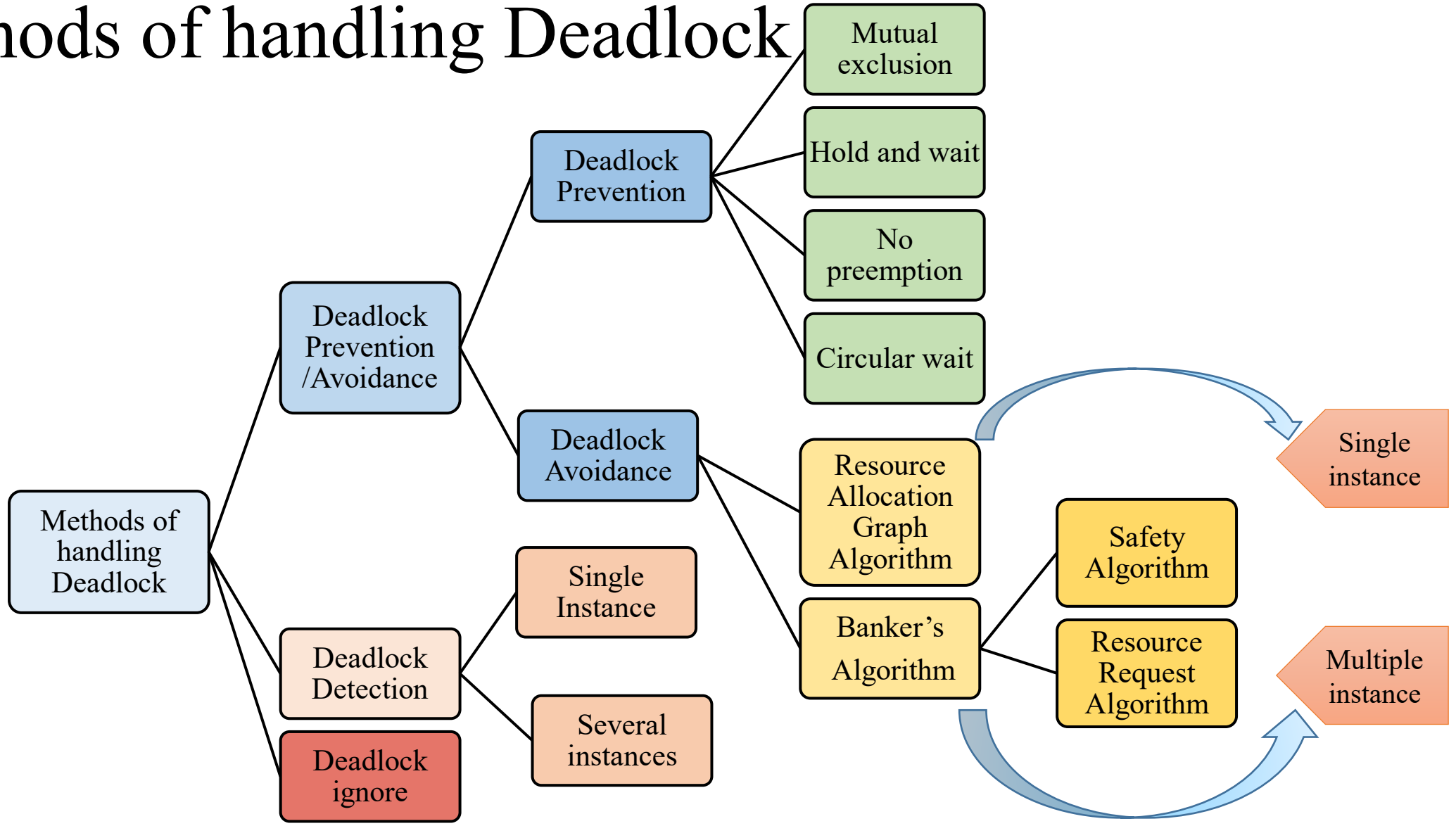
Reference book

- Operating System Concepts by Avi Silberschatz (9th edition)
- Chapter -7

Outline


Deadlock
System Model
Deadlock Characterization
Resource-Allocation Graph
Methods of handling Deadlock

Methods of handling Deadlock



Deadlock

- A process requests resources; if the resources are not available at that time, the process enters a waiting state. Sometimes, a waiting process is never again able to change state, because the resources it has requested are held by other **waiting** processes. This situation is called a deadlock.



System Model

- A system consists of a finite number of resources to be distributed among a number of competing processes. The resources may be partitioned into several types (or classes), each consisting of some number of identical **instances**. CPU cycles, files, and I/O devices (such as printers and DVD drives) are examples of resource types.
- A process may request as many resources as it requires to carry out its designated task. Obviously, the number of resources requested may not exceed the total number of resources available in the system

System Model

- Under the normal mode of operation, a process may utilize a resource in only the following sequence:

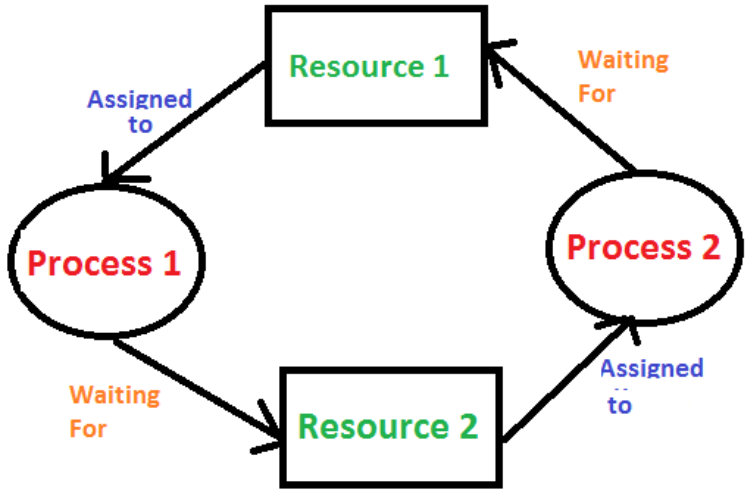
  1. Request. The process requests the resource. If the request cannot be granted immediately (for example, if the resource is being used by another process), then the requesting process must wait until it can acquire the resource.

  2. Use. The process can operate on the resource (for example, if the resource is a printer, the process can print on the printer).

  3. Release. The process releases the resource.

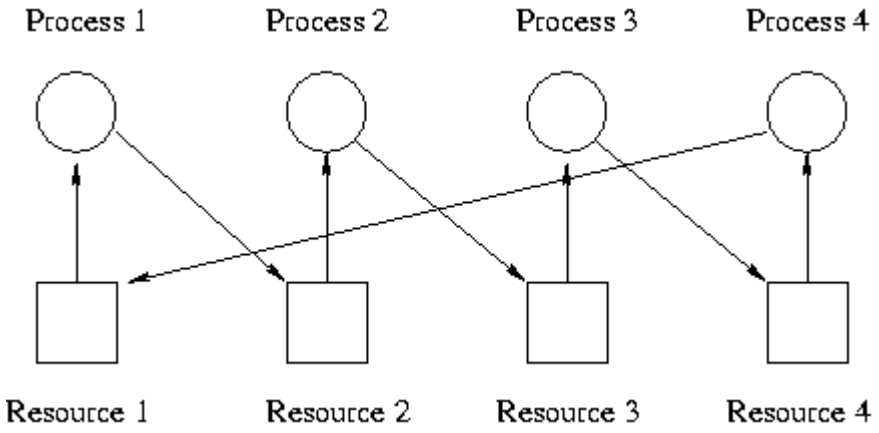
System Model

- A set of processes is in a deadlocked state when every process in the set is waiting for an event that can be caused only by another process in the set.



System Model

- A set of processes is in a deadlocked state when every process in the set is waiting for an event that can be caused only by another process in the set.



Deadlock Characterization: (Necessary Conditions)

- In a deadlock, processes never finish executing, and system resources are tied up, preventing other jobs from starting. A deadlock situation can arise if the following four conditions hold simultaneously in a system:
  1. **Mutual exclusion**. At least one resource must be held in a non sharable mode; that is, only one process at a time can use the resource. If another process requests that resource, the requesting process must be delayed until the resource has been released.

  2. **Hold and wait**. A process must be holding at least one resource and waiting to acquire additional resources that are currently being held by other processes.

Necessary Conditions

3. **No preemption**. Resources cannot be preempted; that is, a resource can be released only voluntarily by the process holding it, after that process has completed its task.

4. **Circular wait**. A set {P0, P1, ..., Pn} of waiting processes must exist such that P0 is waiting for a resource held by P1, P1 is waiting for a resource held by P2, ..., Pn−1 is waiting for a resource held by Pn, and Pn is waiting for a resource held by P0.

We emphasize that all four conditions must hold for a deadlock to occur.

Resource-Allocation Graph

- Deadlocks can be described more precisely in terms of a directed graph called a system resource-allocation graph. This graph consists of a set of vertices V and a set of edges E.

❑The set of vertices V is partitioned into two different types of nodes:

- P = {P1, P2, ..., Pn}, the set consisting of all the active processes in the system,

&

- R = {R1, R2, ..., Rm}, the set consisting of all resource types in the system

Resource-Allocation Graph

- A directed edge from process Pi to resource type Rj is denoted by Pi → Rj ;it signifies that process Pi has requested an instance of resource type Rj and is currently waiting for that resource
- **Request edge:** Pi → Rj (it signifies that process Pi has requested an instance of resource type Rj and is currently waiting for that resource)
- **Assignment edge:** Rj → Pi (it signifies that an instance of resource type Rj has been allocated to process Pi)
- Pictorially, we represent each process as a **circle** and each resource type as a **rectangle**.
- Since resource type may have more than one instance, we represent each such instance as a **dot** within the **rectangle**.

Resource-Allocation Graph

- R = {R1, R2, R3, R4}
- E = {P1 → R1, P2 → R3, R1 → P2, R2 → P2,
  R2 → P1, R3 → P3}
- Resource instances:
  - One instance of resource type R1
  - Two instances of resource type R2
  - One instance of resource type R3
  - Three instances of resource type R4

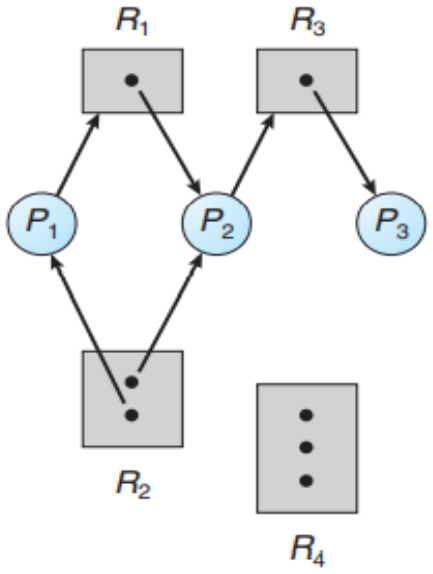


Figure 7.1 Resource-allocation graph.

Resource-Allocation Graph

- Process states:
  - Process P1 is holding an instance of resource type R2 and is waiting for an instance of resource type R1.
  - Process P2 is holding an instance of R1 and an instance of R2 and is waiting for an instance of R3.
  - Process P3 is holding an instance of R3

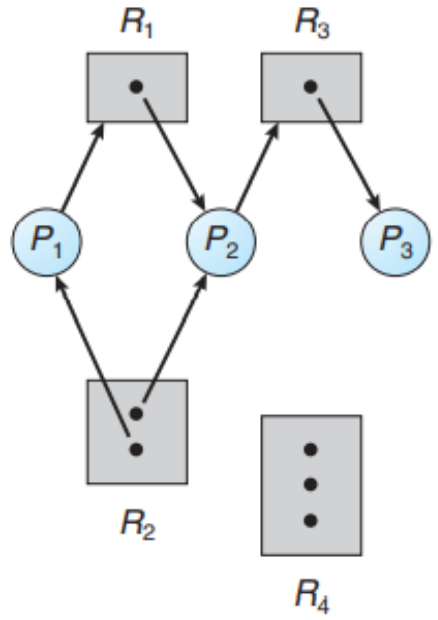


Figure 7.1 Resource-allocation graph.

Resource-Allocation Graph

- If the graph contains no cycles, then no process in the system is deadlocked. If the graph does contain a cycle, then a deadlock **may** exist.
- If the cycle involves only a set of resource types, each of which has only a single instance, then a deadlock has occurred. Each process involved in the cycle is deadlocked. In this case, a cycle in the graph **is both a necessary and a sufficient condition** for the existence of deadlock.
- If each resource type has several instances, then a cycle does not necessarily imply that a deadlock has occurred. In this case, a cycle in the graph is a **necessary but not a sufficient condition** for the existence of deadlock.

Resource-Allocation Graph

- Resource-Allocation Graph: Two cycles

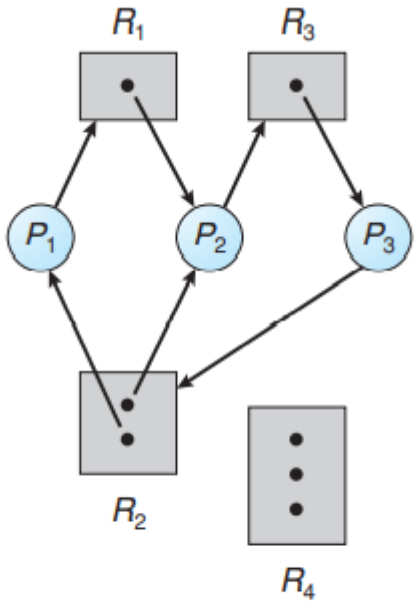


Figure 7.2 Resource-allocation graph with a deadlock.

Resource-Allocation Graph

- Resource-Allocation Graph: Two cycles

$$P_1 \rightarrow R_1 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$$
$$P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_2$$

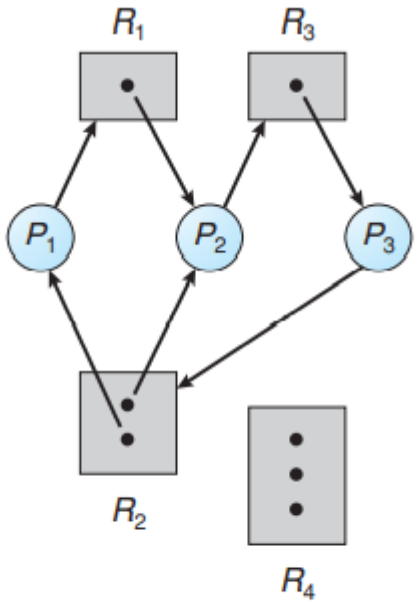


Figure 7.2 Resource-allocation graph with a deadlock.

- Processes P1, P2, and P3 are deadlocked.

Resource-Allocation Graph

- Resource-Allocation Graph:

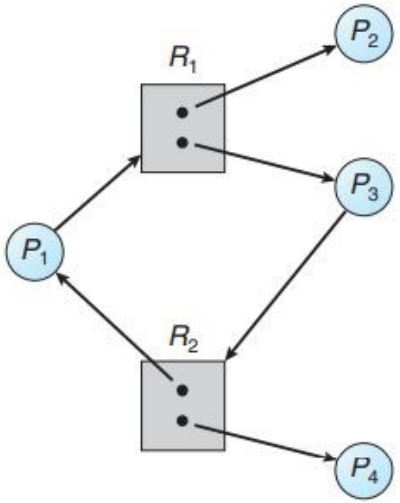


Figure 7.3 Resource-allocation graph with a cycle but no deadlock.

Resource-Allocation Graph

- Resource-Allocation Graph: one cycle

$$P_1 \rightarrow R_1 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$$

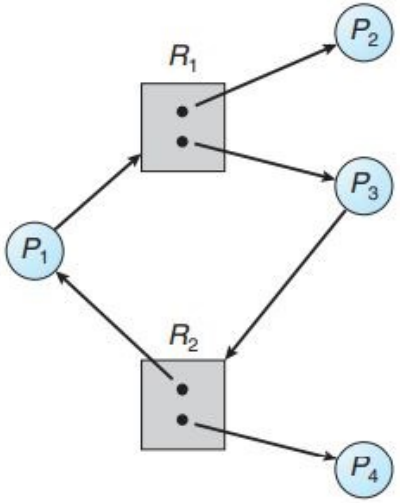


Figure 7.3 Resource-allocation graph with a cycle but no deadlock.

Resource-Allocation Graph

- Resource-Allocation Graph: one cycle

$$P_1 \rightarrow R_1 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$$

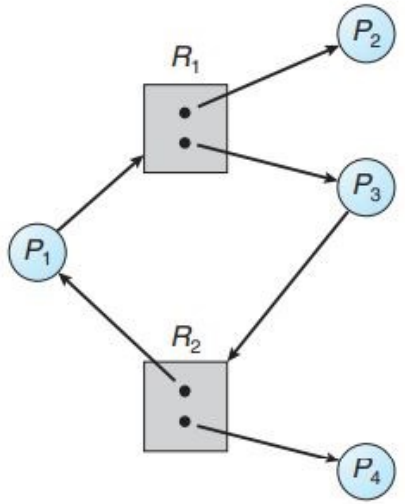


Figure 7.3 Resource-allocation graph with a cycle but no deadlock.

- However, there is no deadlock. Observe that process P4 may release its instance of resource type R2. That resource can then be allocated to P3, breaking the cycle.

Methods of handling Deadlock

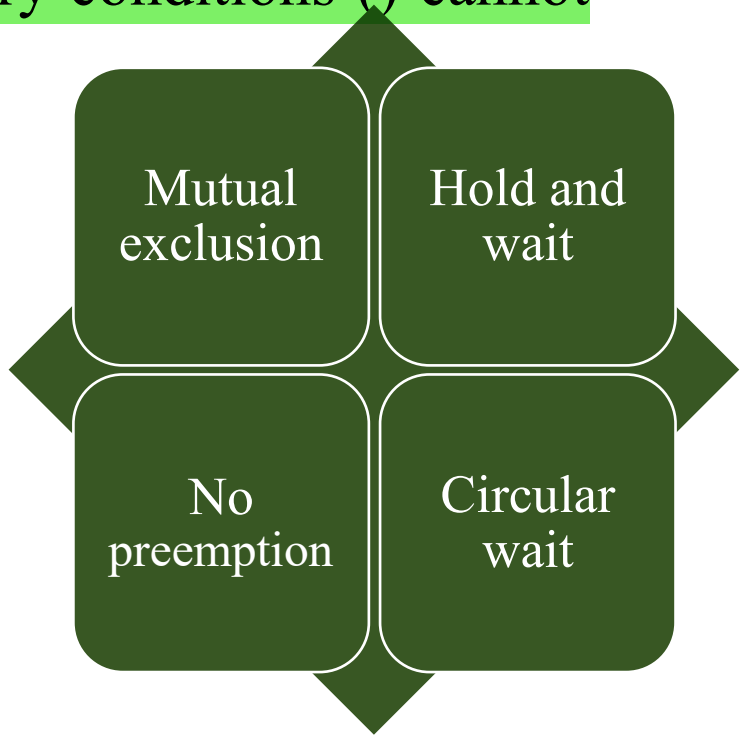
- Generally speaking, we can deal with the deadlock problem in one of three ways:

1. We can use a protocol to **prevent** or **avoid** deadlocks, ensuring that the system will never enter a deadlocked state.
2. We can allow the system to enter a deadlocked state, **detect** it, and recover.
3. We can **ignore** the problem altogether and pretend that deadlocks never occur in the system.

The third solution is the one used by most operating systems, including Linux and Windows. It is then up to the application developer to write programs that handle deadlocks.

Methods of handling Deadlock

- 1.To ensure that deadlocks never occur, the system can use either a deadlock **prevention** or a deadlock**-avoidance** scheme. Deadlock prevention provides a set of methods to ensure that at least one of the necessary conditions () cannot hold.



Methods of handling Deadlock

2.

- If a system does not employ either a deadlock-prevention or a deadlock avoidance algorithm, then a deadlock situation may arise.
- In this environment, the system can provide an algorithm that examines the state of the system to **determine** whether a deadlock has occurred and an algorithm to **recover** from the deadlock (if a deadlock has indeed occurred).

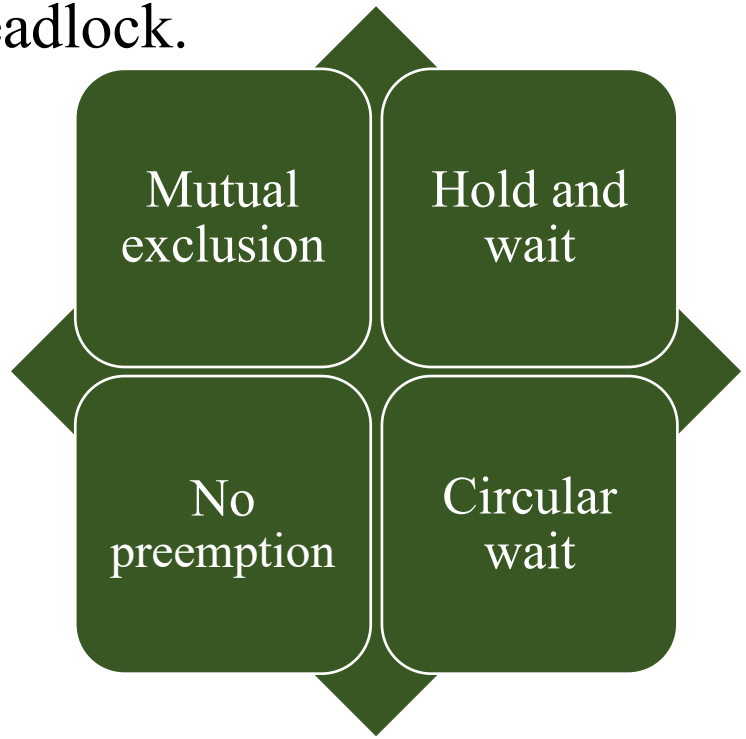
Methods of handling Deadlock

3.

- In the absence of algorithms to detect and recover from deadlocks, we may arrive at a situation in which the system is in a deadlocked state yet has no way of recognizing what has happened. In this case, the undetected deadlock will cause the system's performance to deteriorate, because resources are being held by processes that cannot run and because more and more processes, as they make requests for resources, will enter a deadlocked state. Eventually, the system will stop functioning and will need to be restarted manually.

Deadlock Prevention

- By ensuring that at least one of these conditions cannot hold, we can prevent the occurrence of a deadlock.



Deadlock Prevention : Mutual exclusion

- **Mutual exclusion:** The mutual exclusion condition must hold for non sharable resources. Read-only files are a good example of a sharable resource. If several processes attempt to open a read-only file at the same time, they can be granted simultaneous access to the file. A process never needs to wait for a sharable resource. Sharable resource do not require mutually exclusive access and thus can not be involved in a deadlock. In general, however, we cannot prevent deadlocks by denying the mutual-exclusion condition, because some resources are intrinsically non-sharable. For example, a printer cannot be simultaneously shared by several processes.

Deadlock Prevention: Hold and wait

- **Hold and wait:** To ensure that the hold-and-wait condition never occurs in the system, we must guarantee that, whenever a process requests a resource, it does not hold any other resources. **One** protocol that we can use requires each process to request and be allocated all its resources before it begins execution. An **alternative** protocol allows a process to request resources only when it has none. A process may request some resources and use them. Before it can request any additional resources, it must release all the resources that it is currently allocated.

Deadlock Prevention: Hold and wait

- **Hold and wait:** To illustrate the difference between these two protocols, we consider a process that copies data from a DVD drive to a file on disk, sorts the file, and then prints the results to a printer. If all resources must be requested at the beginning of the process, then the process must initially request the DVD drive, disk file, and printer. It will hold the printer for its entire execution, even though it needs the printer only at the end.
- The second method allows the process to request initially only the DVD drive and disk file. It copies from the DVD drive to the disk and then releases both the DVD drive and the disk file. The process must then request the disk file and the printer. After copying the disk file to the printer, it releases these two resources and terminates.

Deadlock Prevention

- Both these protocols have two main disadvantages. **First**, resource utilization may be low, since resources may be allocated but unused for a long period. In the example given, for instance, we can release the DVD drive and disk file, and then request the disk file and printer, only if we can be sure that our data will remain on the disk file. Otherwise, we must request all resources at the beginning for both protocols.
- **Second**, starvation is possible. A process that needs several popular resources may have to wait indefinitely, because at least one of the resources that it needs is always allocated to some other process.

Deadlock Prevention: No preemption

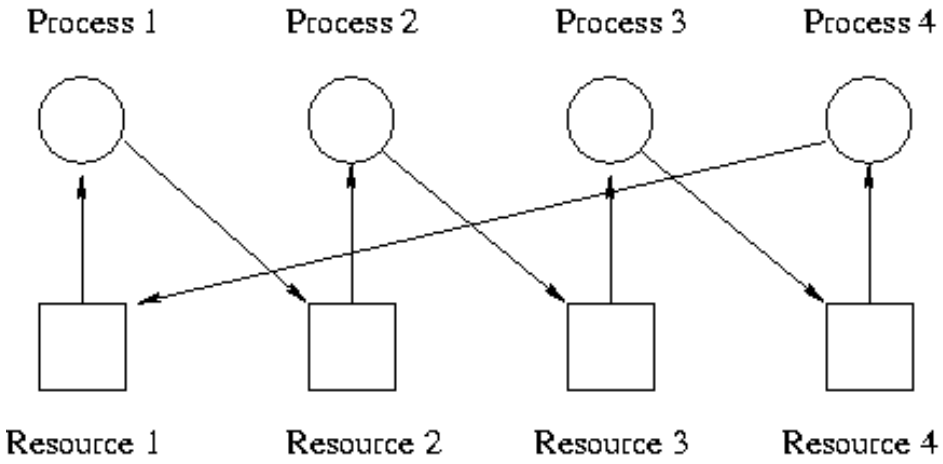
- **No preemption:** The third necessary condition for deadlocks is that there be no preemption of resources that have already been allocated. To ensure that this condition does not hold, we can use the following protocol.
- If a process is holding some resources and requests another resource that cannot be immediately allocated to it (that is, the process must wait), then all resources the process is currently holding are preempted. In other words, these resources are implicitly released. The preempted resources are added to the list of resources for which the process is waiting. The process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting.

Deadlock Prevention: No preemption

- **No preemption: Alternatively**, if a process requests some resources, we first check whether they are available. If they are, we allocate them. If they are not, we check whether they are allocated to some other process that is waiting for additional resources. If so, we preempt the desired resources from the waiting process and allocate them to the requesting process. If the resources are neither available nor held by a waiting process, the requesting process must wait. While it is waiting, some of its resources may be preempted, but only if another process requests them. A process can be restarted only when it is allocated the new resources it is requesting and recovers any resources that were preempted while it was waiting.
- This protocol is often applied to resources whose state can be easily saved and restored later, such as CPU registers and memory space. It cannot generally be applied to such resources as printers and tape drives.

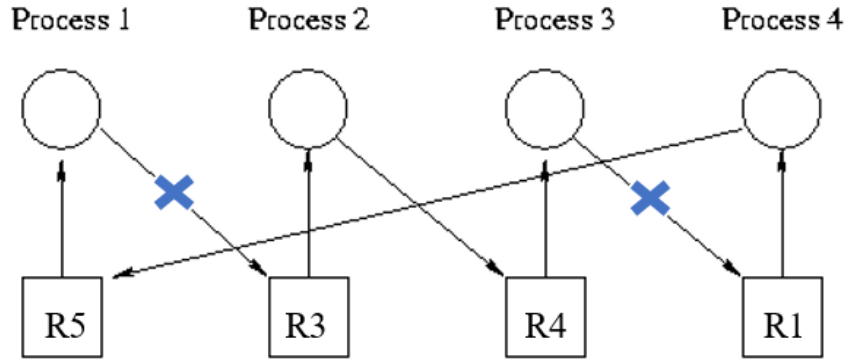
Deadlock Prevention: Circular wait

- **Circular wait:** One way to ensure that this condition never holds is to impose a total ordering of all resource types and to require that each process requests resources in an increasing order of enumeration.



Deadlock Prevention: Circular wait

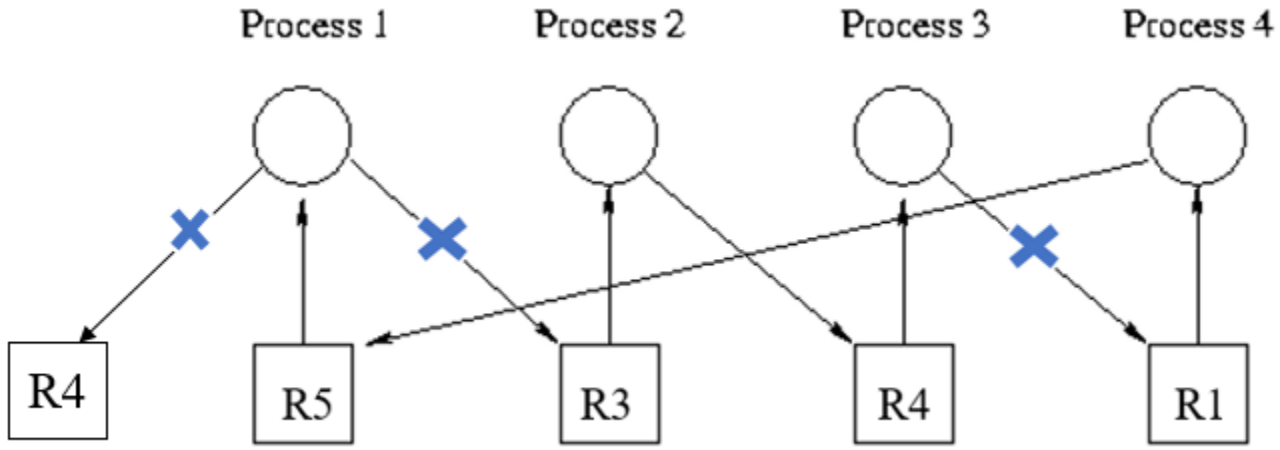
- **Example**: Suppose process P1 is allocated resource R5. Now if P1 requests for Resources R4 and R3 (which are lesser than R5), such requests will not be granted. Only request for Resource greater than R5 will be granted.



- Developing an ordering, or hierarchy, does not in itself prevent deadlock. It is up to application developers to write programs that follow the ordering.

Deadlock Prevention: Circular wait

- **Example**: Suppose process P1 is allocated resource R5. Now if P1 requests for Resources R4 and R3 (which are lesser than R5), such requests will not be granted. Only request for Resource greater than R5 will be granted.



Deadlock Avoidance

- Safe State
- Resource-Allocation-Graph Algorithm (For Single Instance)
- Banker's Algorithm (For Multiple Instances)
  - Safety Algorithm
  - Resource-Request Algorithm

Deadlock Avoidance

- An alternative method for avoiding deadlocks is to require additional information about how resources are to be requested.
- For example, in a system with one tape drive and one printer, the system might need to know that process P will request first the tape drive and then the printer before releasing both resources, whereas process Q will request first the printer and then the tape drive.

Deadlock Avoidance

- With this knowledge of the complete sequence of requests and releases for each process, the system can decide for each request whether or not the process should wait in order to avoid a possible future deadlock.
- Each request requires that in making this decision the system consider the resources currently available, the resources currently allocated to each process, and the future requests and releases of each process.

Deadlock Avoidance (Safe State)

- A state is safe if the system can allocate resources to each process (up to its maximum) in some order and still avoid a deadlock.
- More formally, a system is in a safe state only if there exists a safe sequence. A sequence of processes <P1, P2, ..., Pn> is a safe sequence for the current allocation state if, for each Pi , the resource requests that Pi can still make can be satisfied by the currently available resources plus the resources held by all Pi.

Safe State

- A safe state is not a deadlocked state. Conversely, a deadlocked state is an unsafe state. Not all unsafe states are deadlocks.

- An unsafe state may lead to a deadlock. As long as the state is safe, the operating system can avoid unsafe (and deadlocked) states. In an unsafe state, the operating system cannot prevent processes from requesting resources in such a way that a deadlock occurs. The behavior of the processes controls unsafe states.

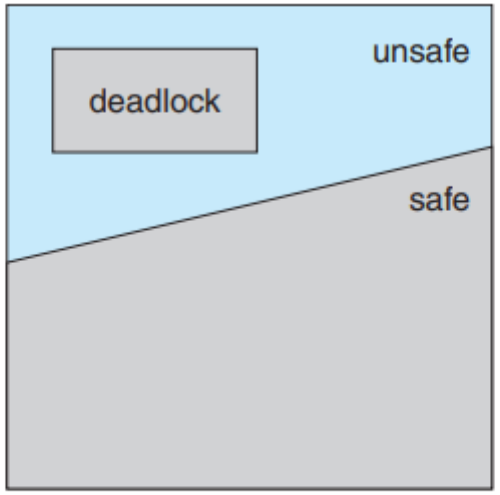


Figure 7.6 Safe, unsafe, and deadlocked state spaces.

Safe State

- To illustrate, we consider a system with **twelve** magnetic tape drives and three processes: P0, P1, and P2.
- Free drive= 12-9=3

	Maximum Needs	Current Needs
P_0	10	5
P_1	4	2
P_2	9	2

- At time t0, the system is in a safe state. The sequence <P1, P0, P2> satisfies the safety condition.

Safe State

- A system can go from a safe state to an unsafe state. Suppose that, at time t1, process P2 requests and is allocated one more tape drive.

	Maximum Needs	Current Needs
P_0	10	5
P_1	4	2
P_2	9	~~2~~ 3

- Free drive= 12-9 = 3-1=2
- P1->….
- The system is no longer in a safe state.

Deadlock Avoidance (Resource-Allocation-Graph Algorithm)

- A claim edge Pi → Rj indicates that process Pi may request resource Rj at some time in the future.
- A claim edge Pi → Rj indicates that process Pi may request resource Rj at some time in the future.
- This edge resembles a request edge in direction but is represented in the graph by a dashed line.

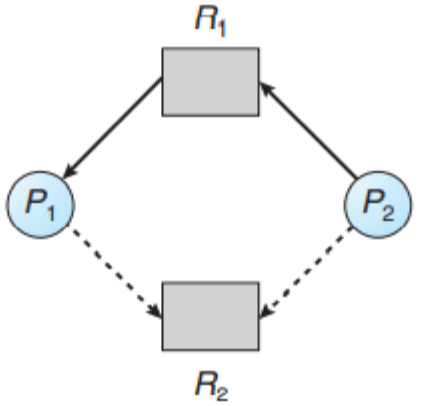


Figure 7.7 Resource-allocation graph for deadlock avoidance.

Deadlock Avoidance (Resource-Allocation-Graph Algorithm)

- When process Pi requests resource Rj , the claim edge Pi → Rj is converted to a request edge.
- Similarly, when a resource Rj is released by Pi , the assignment edge Rj → Pi is reconverted to a claim edge Pi → Rj .

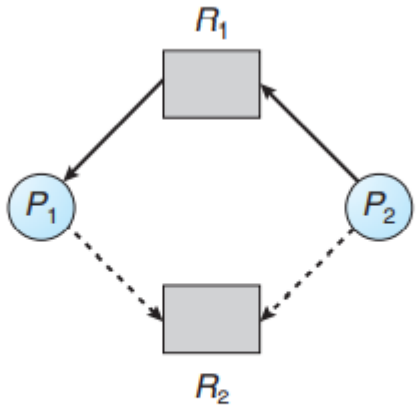


Figure 7.7 Resource-allocation graph for deadlock avoidance.

Resource-Allocation-Graph Algorithm

- The resources must be claimed a priori in the system. That is, before process Pi starts executing, all its claim edges must already appear in the resource-allocation graph.
- Now suppose that process Pi requests resource Rj . The request can be granted only if converting the request edge Pi → Rj to an assignment edge Rj → Pi does not result in the formation of a cycle in the resource-allocation graph. We check for safety by using a cycle-detection algorithm.
- If no cycle exists, then the allocation of the resource will leave the system in a safe state. If a cycle is found, then the allocation will put the system in an unsafe state. In that case, process Pi will have to wait for its requests to be satisfied.

Resource-Allocation-Graph Algorithm

- To illustrate this algorithm, we consider the resource-allocation graph of Figure 7.7. Suppose that P2 requests R2. Although R2 is currently free, we cannot allocate it to P2, since this action will create a cycle in the graph (Figure 7.8). A cycle, as mentioned, indicates that the system is in an unsafe state. If P1 requests R2, and P2 requests R1, then a deadlock will occur.

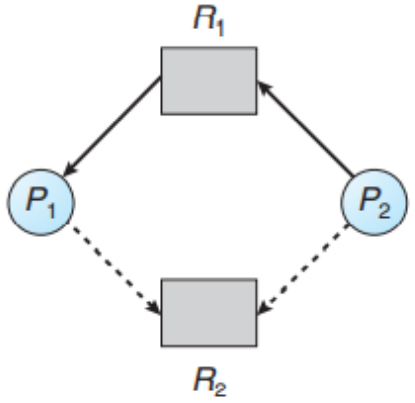


Figure 7.7 Resource-allocation graph for deadlock avoidance.

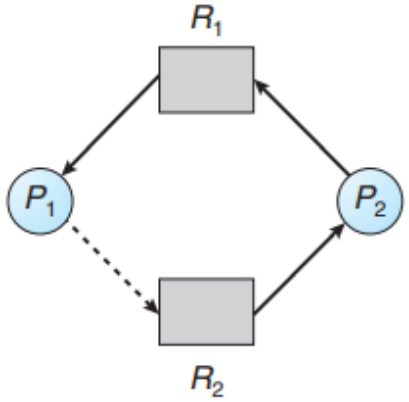


Figure 7.8 An unsafe state in a resource-allocation graph.

Resource-Allocation-Graph Algorithm

- This algorithm is only for those resources where there is only one instance of a particular resource. The resource-allocation-graph algorithm is not applicable to a resource allocation system with multiple instances of each resource type.

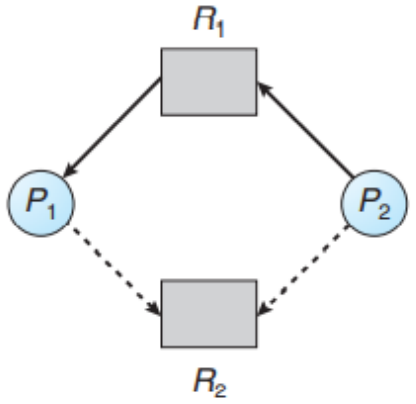


Figure 7.7 Resource-allocation graph for deadlock avoidance.

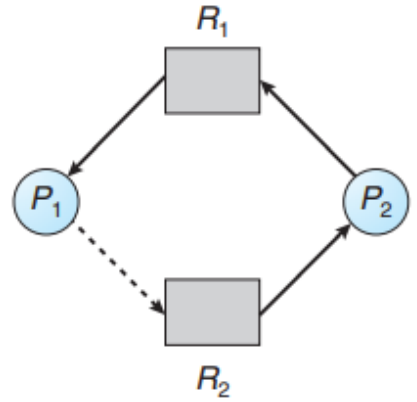


Figure 7.8 An unsafe state in a resource-allocation graph.

Banker's Algorithm

- An algorithm that can be used for deadlock in resource allocation systems with **multiple instances** of each resource type.
- It is less efficient than the resource-allocation graph scheme. This algorithm is commonly known as the **banker's algorithm**. The name was chosen because the algorithm could be used in a banking system to ensure that the bank never allocated its available cash in such a way that it could no longer satisfy the needs of all its customers.
- The resources should not be allocated in such a way that the need of the process can not be satisfied.

Banker's Algorithm

- Let **n** is the number of **processes** in the system and **m** is the number of **resource** types.
  - **Available**. A vector of length m indicates the number of available resources of each type. If Available[j] equals k, then k instances of resource type Rj are available.
  - **Max**. An n × m matrix defines the maximum demand of each process. If Max[i][j] equals k, then process Pi may request at most k instances of resource type Rj .
  - **Allocation**. An n × m matrix defines the number of resources of each type currently allocated to each process. If Allocation[i][j] equals k, then process Pi is currently allocated k instances of resource type Rj.

Banker's Algorithm

- **Need**. An n × m matrix indicates the remaining resource need of each process. If Need[i][j] equals k, then process Pi may need k more instances of resource type Rj to complete its task. Note that
  - Need[i][j] equals Max[i][j] − Allocation[i][j].

- Allocation_i specifies the resources currently allocated to process Pi.
- The vector Need_i specifies the additional resources that process Pi may still request to complete its task.

Banker's Algorithm

- Two types of Banker's Algorithm:
  - ❑Safety algorithm
  - ❑Resource request algorithm

Safety Algorithm

- We can now present the algorithm for finding out whether or not a system is in a safe state. This algorithm can be described as follows:
  1. Let Work and Finish be vectors of length m and n, respectively. Initialize Work = Available and Finish[i] = false for i = 0, 1, ..., n − 1.
  2. Find an index i such that both
     a) Finish[i] == false
     b) Needi ≤ Work

     If no such i exists, go to step 4.
  3. Work = Work + Allocation

     Finish[i] = true

     Go to step 2.
  4. If Finish[i] == true for all i, then the system is in a safe state.

Example of Safety Algorithm

- Consider a system with five processes P0 through P4 and three resource types A, B, and C. Resource type A has **ten** instances, resource type B has **five** instances, and resource type C has **seven** instances. Suppose that, at time T0, the following snapshot of the system has been taken:
- Matrix Need =Max − Allocation

	Allocation	*Max*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

	Need
	A B C
P_0	7 4 3
P_1	1 2 2
P_2	6 0 0
P_3	0 1 1
P_4	4 3 1

Example of Safety Algorithm

- We claim that the system is currently in a safe state.

	Allocation	*Max*	*Available*		*Need*
	A B C	*A B C*	*A B C*		*A B C*
P_0	0 1 0	7 5 3	3 3 2	P_0	7 4 3
P_1	2 0 0	3 2 2		P_1	1 2 2
P_2	3 0 2	9 0 2		P_2	6 0 0
P_3	2 1 1	2 2 2		P_3	0 1 1
P_4	0 0 2	4 3 3		P_4	4 3 1

- The sequence <P1, P3, P4, P2, P0> satisfies the safety criteria.

Resource-Request Algorithm

- Here, we describe the algorithm for determining whether requests can be safely granted.
- Suppose now that process P1 requests one additional instance of resource type A and two instances of resource type C, so Request1 = (1,0,2).
- To decide whether this request can be immediately granted, we first check that Request1 ≤ Available—that is, that (1,0,2) ≤ (3,3,2), which is true (if not it can not be granted). We then pretend that this request has been fulfilled, and we arrive at the following new state.

Resource-Request Algorithm

- We must determine whether this new system state is safe. To do so, we execute our safety algorithm and find that the sequence **<P1, P3, P4, P0, P2>** satisfies the safety requirement. Hence, we can immediately grant the request of process P1.

	Allocation	*Need*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

Resource-Request Algorithm

- You should be able to see, however, that when the system is in this state, a request for (3,3,0) by P4 cannot be granted, since the resources are not available. Furthermore, a request for (0,2,0) by P0 cannot be granted, even though the resources are available, since the resulting state is unsafe.

	Allocation	*Need*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

Deadlock Detection

- Single Instance of Each Resource Type
- Several Instances of a Resource Type

Deadlock Detection

- If a system does not employ either a deadlock-prevention or a deadlock avoidance algorithm, then a deadlock situation may occur. In this environment, the system may provide:
  - An algorithm that examines the state of the system to determine whether a deadlock has occurred
  - An algorithm to recover from the deadlock

Deadlock Detection: wait-for graph

- If all resources have only a **single** instance, then we can define a deadlock detection algorithm that uses a variant of the resource-allocation graph, called a wait-for graph. We obtain this graph from the resource-allocation graph by removing the resource nodes and collapsing the appropriate edges.

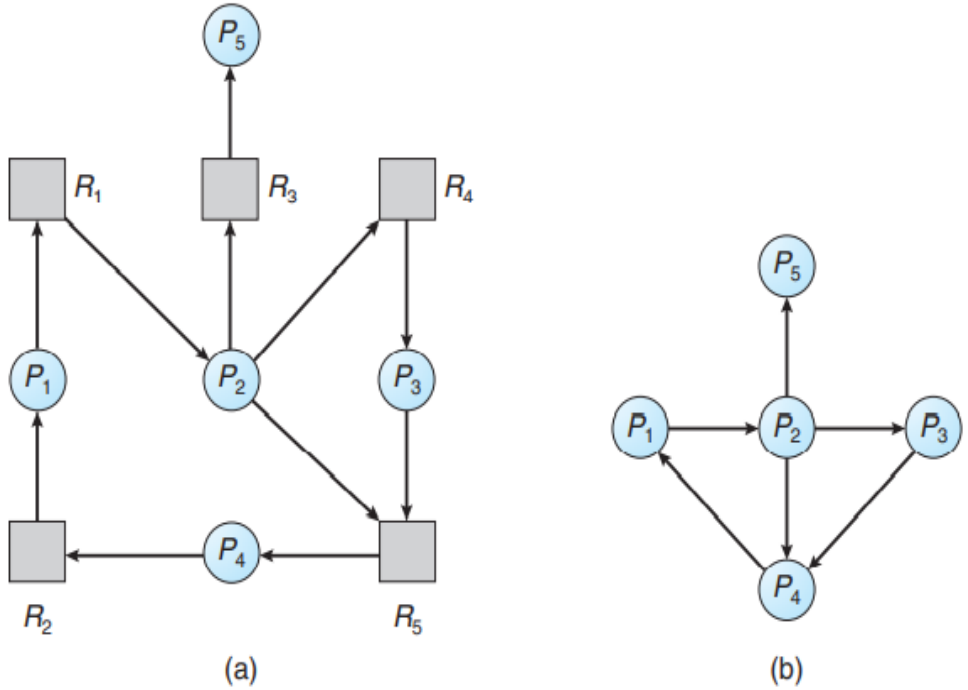


Figure 7.9 (a) Resource-allocation graph. (b) Corresponding wait-for graph.

Deadlock Detection: wait-for graph

- More precisely, an edge from Pi to Pj in a wait-for graph implies that process Pi is waiting for process Pj to release a resource that Pi needs.
- An edge Pi → Pj exists in a wait-for graph if and only if the corresponding resource allocation graph contains two edges Pi → Rq and Rq → Pj for some resource Rq .
- In Figure 7.9, we present a resource-allocation graph and the corresponding wait-for graph.

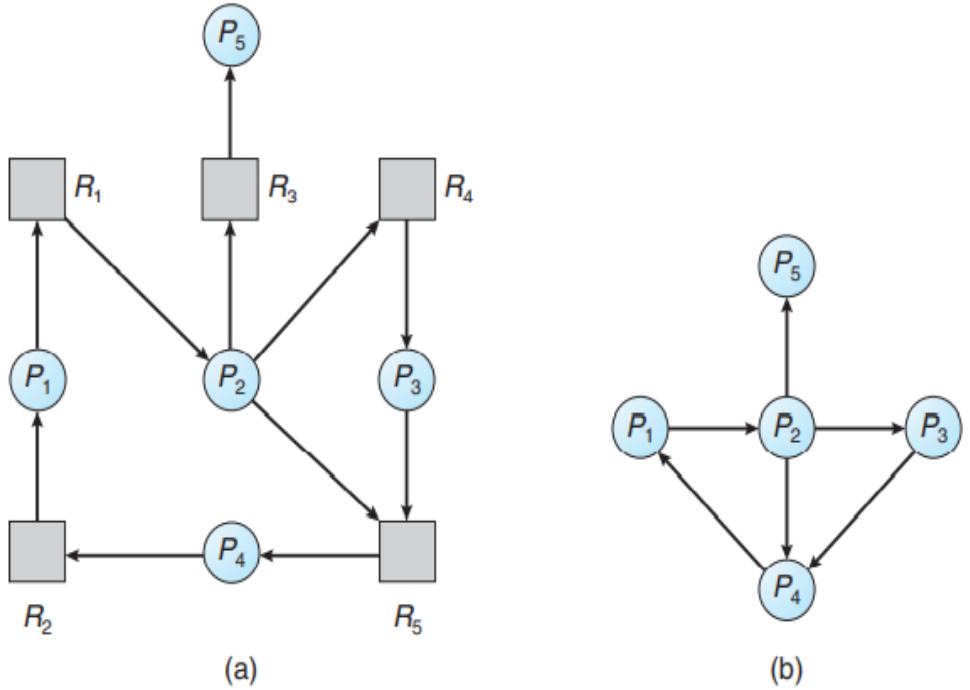


Figure 7.9 (a) Resource-allocation graph. (b) Corresponding wait-for graph.

Deadlock Detection: wait-for graph

- As before, a deadlock exists in the system if and only if the wait-for graph contains a **cycle**. To detect deadlocks, the system needs to maintain the wait for graph and periodically invoke an algorithm that searches for a cycle in the graph.

Several Instances of a Resource Type

- The wait-for graph scheme is not applicable to a resource-allocation system with multiple instances of each resource type. We turn now to a deadlock-detection algorithm that is applicable to such a system. The algorithm employs several time-varying data structures that are similar to those used in the banker's algorithm (Section 7.5.3):

Several Instances of a Resource Type

- Available. A vector of length m indicates the number of available resources of each type.
- Allocation. An n × m matrix defines the number of resources of each type currently allocated to each process.
- Request. An n × m matrix indicates the current request of each process. If Request[i][j] equals k, then process Pi is requesting k more instances of resource type Rj .

Several Instances of a Resource Type

1. Let Work and Finish be vectors of length m and n, respectively. Initialize Work = Available. For i = 0, 1, ..., n–1, if Allocationi ≠ 0, then Finish[i] = false. Otherwise, Finish[i] = true.
2. Find an index i such that both
   a. Finish[i] == false
   b. Requesti ≤ Work
   If no such i exists, go to step 4.
3. Work = Work + Allocationi
   Finish[i] = true
   Go to step 2.
4. If Finish[i] == false for some i, 0 ≤ i < n, then the system is in a deadlocked state. Moreover, if Finish[i] == false, then process Pi is deadlocked.

Several Instances of a Resource Type

- You may wonder why we reclaim the resources of process Pi (in step 3) as soon as we determine that Requesti ≤ Work (in step 2b). We know that Pi is currently not involved in a deadlock (since Requesti ≤ Work). Thus, we take an optimistic attitude and assume that Pi will require no more resources to complete its task; it will thus soon return all currently allocated resources to the system. If our assumption is incorrect, a deadlock may occur later. That deadlock will be detected the next time the deadlock-detection algorithm is invoked.

Several Instances of a Resource Type

- To illustrate this algorithm, we consider a system with five processes P0 through P4 and three resource types A, B, and C. Resource type A has seven instances, resource type B has two instances, and resource type C has six instances. Suppose that, at time T0, we have the following resource-allocation state:

	Allocation	***Request***	***Available***
	A B C	*A B C*	*A B C*
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

Several Instances of a Resource Type

- We claim that the system is not in a deadlocked state. Indeed, if we execute our algorithm,
- We will find that the sequence <P0, P2, P3, P1, P4>/ <P0, P2, P3, P4, P1> results in Finish[i] == true for all i.

	Allocation	*Request*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

Several Instances of a Resource Type

- Suppose now that process P2 makes one additional request for an instance of type C. The Request matrix is modified as follows:

	Allocation	Request	Available
	A B C	A B C	A B C
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	



	Request
	A B C
P_0	0 0 0
P_1	2 0 2
P_2	0 0 1
P_3	1 0 0
P_4	0 0 2

Several Instances of a Resource Type

- We claim that the system is now deadlocked. Although we can reclaim the resources held by process P0, the number of available resources is not sufficient to fulfill the requests of the other processes. Thus, a deadlock exists, consisting of processes P1, P2, P3, and P4.

	Allocation	*Request*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	



	Request
	A B C
P_0	0 0 0
P_1	2 0 2
P_2	0 0 1
P_3	1 0 0
P_4	0 0 2

Detection-Algorithm Usage

- When should we invoke the detection algorithm? The answer depends on two factors:

1. How often is a deadlock likely to occur?
2. How many processes will be affected by deadlock when it happens?

- If deadlocks occur frequently, then the detection algorithm should be invoked frequently. Deadlocks occur only when some process makes a request that cannot be granted immediately. This request may be the final request that completes a chain of waiting processes. In the extreme, then, we can invoke the deadlock detection algorithm every time a request for allocation cannot be granted immediately. In this case, we can identify not only the deadlocked set of processes but also the specific process that "caused" the deadlock.

Detection-Algorithm Usage

- Invoking the deadlock-detection algorithm for every resource request will incur considerable overhead in computation time. A less expensive alternative is simply to invoke the algorithm at defined intervals—for example, once per hour or whenever CPU utilization drops below 40 percent. (A deadlock eventually cripples system throughput and causes CPU utilization to drop.) If the detection algorithm is invoked at arbitrary points in time, the resource graph may contain many cycles. In this case, we generally cannot tell which of the many deadlocked processes “caused” the deadlock.

Recovery from Deadlock

- Process Termination
- Resource Preemption

Recovery from Deadlock

- When a detection algorithm determines that a deadlock exists, several alternatives are available.
  - One possibility is to inform the operator that a deadlock has occurred and to let the operator deal with the deadlock manually.
  - Another possibility is to let the system recover from the deadlock automatically.
- There are two options for breaking a deadlock.
  - abort one or more processes to break the circular wait.
  - preempt some resources from one or more of the deadlocked processes.

Process Termination

- To eliminate deadlocks by aborting a process, we use one of two methods. In both methods, the system reclaims all resources allocated to the terminated processes.
  - **Abort all deadlocked processes**. This method clearly will break the deadlock cycle, but at great expense. The deadlocked processes may have computed for a long time, and the results of these partial computations must be discarded and probably will have to be recomputed later.
  - **Abort one process at a time until the deadlock cycle is eliminated**. This method incurs considerable overhead, since after each process is aborted, a deadlock-detection algorithm must be invoked to determine whether any processes are still deadlocked.

Process Termination

Aborting a process may not be easy. Some examples:

- If the process was in the midst of updating a file, terminating it will leave that file in an incorrect state.
- If the process was in the midst of printing data on a printer, the system must reset the printer to a correct state before printing the next job.

Process Termination

If the partial termination method is used, then

- We must determine which deadlocked process (or processes) should be terminated.
- This determination is a policy decision, similar to CPU-scheduling decisions. The question is basically an economic one; we should abort those processes whose termination will incur the **minimum cost.**

Process Termination

- **Many factors may affect which process is chosen, including:**
  1. What the priority of the process is.
  2. How long the process has computed and how much longer the process will compute before completing its designated task.
  3. How many and what types of resources the process has used (for example, whether the resources are simple to preempt).
  4. How many more resources the process needs in order to complete.
  5. How many processes will need to be terminated.
  6. Whether the process is interactive or batch.

Resource Preemption

- Here, we successively preempt some resources from processes and give these resources to other processes until the deadlock cycle is broken.
- If preemption is required to deal with deadlocks, then three issues need to be addressed:
  - Selecting a victim.
  - Rollback
  - Starvation

Resource Preemption: Selecting a victim

Selecting a victim:

- Which resources and which processes are to be preempted? As in process termination, we must determine the order of preemption to minimize cost.
- Cost factors may include such parameters as the number of resources a deadlocked process is holding and the amount of time the process has thus far consumed.

Resource Preemption: Rollback

Rollback:

If we preempt a resource from a process, what should be done with that process?

- it cannot continue with its normal execution; it is missing some needed resource.
- We must roll back the process to some safe state and restart it from that state. Since, in general, it is difficult to determine what a safe state is, the simplest solution is a total rollback: abort the process and then restart it. Although it is more effective to roll back the process only as far as necessary to break the deadlock, this method requires the system to keep more information about the state of all running processes.

Resource Preemption: Starvation

Starvation:

- How do we ensure that starvation will not occur?
- That is, how can we guarantee that resources will not always be preempted from the same process?
- In a system where victim selection is based primarily on cost factors, it may happen that the same process is always picked as a victim. As a result, this process never completes its designated task, a starvation situation any practical system must address. Clearly, we must ensure that a process can be picked as a victim only a (small) finite number of times. The most common solution is to include the number of rollbacks in the cost factor.